Assignment 2

ENGE X200 Microcontroller Programming
Adelaide University

Total Marks: 100 marks (20% of overall grade)

Due Date: Tuesday 3 November 2026, 11.59 PM via myLearning

Submission Details: Please submit the following **two files** on my learning: a zip file containing your code and results video named **student_id.zip** (i.e. a1234567.zip), and your report pdf (a1234567.pdf) on myLearning (word count: 1500-2000 words). All code files submitted will be checked for functionality, so ensure you upload every file required to execute your program via VS Code and the Wokwi Emulator. Failure to do so will result in a loss of marks.

Assignment Brief

Modern aircraft utilise fly-by-wire technology to command flight control surfaces on the wings and tail to steer and stabilise the aircraft in the sky. To assist with manipulation; pilots primarily rely on flight displays to interpret information on critical flight parameters such as altitude, airspeed, attitude and structural warnings. As a result, ensuring the plane maintains operation within safety margins is entirely reliant on a pilot's reasoning based on visual feedback and subsequent commands they send via the control stick.

Haptics is concerned with the science of touch; pertaining to both kinaesthetic (force) and tactile sensations which are pertinent to how humans perceive and interact with their physical surrounds. Within the realm of orthopaedic robotics, to ensure bone-milling procedures are conducted within the specified surgical site, virtual force fields are implemented at the boundaries. If the surgeon commands the robot to penetrate beyond this boundary, an immediate force is reflected back to them with their commanding device and the haptic feedback used as a danger warning.

A leading aircraft manufacturer has tasked you with developing a similar force-feedback system for an aircraft cruising at altitude. Specifically, you are tasked with writing embedded code for a haptic system which is to be implemented on a `Nucleo C031C6` development board that utilises a `STM32C031C6` microcontroller. Your code must

1. Enforce individual haptic feedback on the pitch (θ) and roll (ϕ) axes of the control stick;
2. Utilise a unified tilt metric based on the current pitch and roll values, i.e. $\alpha = \sqrt{\theta^2 + \phi^2}$, to provide additional force feedback on the aircraft's current flight envelope state on a separate control stick;
3. Letting γ be a placeholder angle for pitch or roll; free motion (i.e. no force) of the control stick in a single axis is experienced in the region of $-30^\circ < \gamma < 30^\circ$; a compliant force is reflected on the control stick between $30^\circ \leq \gamma < 50^\circ$ and $-50^\circ < \gamma \leq -30^\circ$; and for $\gamma > 50^\circ$ and $\gamma < -50^\circ$, an impenetrable force clamps the position of the control stick to $\pm 50^\circ$, i.e. a control stick command of 60° will be set at 50°.
4. For, the unified tilt metric $\alpha \geq 0$ leading to free motion $0^\circ \leq \alpha < 20^\circ$; compliant force in the region of $20^\circ < \alpha < 70^\circ$ and impenetrable clamping at $\alpha \geq 70^\circ$. This force feedback instigates earlier as a

warning that both roll and pitch are away from level position but not enough to trigger their individual axes, acting as an overall indicator;

5. If both the pitch and roll axes are simultaneously in their respective impenetrable regions; auto-pilot must automatically be triggered;
6. To safely transition from autopilot to manual mode, a pushbutton in the cockpit must be pressed and both pitch and roll of the control signal be in their free motion range. When operating in manual mode, pressing the autopilot pushbutton will return the system to autopilot mode; and
7. Separate LEDs are illuminated to indicate if the aircraft is in autopilot or manual mode.

Note, these angle bounds are exaggerated for the sake of having visually discernible results in Wokwi and in a real-world scenario the limits would be much lower.

Simulation Setup

The scenario described in the previous section is to be simulated using the Wokwi emulator. To physically implement haptic control, torque-controlled DC motors (brushless or brushed) are typically employed. Wokwi itself does not have emulation of either type of these motors; hence, servo motors are used in their place. The caveat to this is that servo motors are positioned controlled. Thus to simulate the force-feedback of the axes described in the brief your servos should:

- Stay at the neutral position (servo is at an angle of 90° for pitch and roll, 0° for tilt) when the commanded axis angle is in the free motion range;
- Track the commanded axis angle in a one-to-one basis when in the compliant region;
- Be clamped to the respective bound when in the impenetrable region;
- Moving in a clockwise direction with respect to the servo setup in Fig 1 is consider to be negative; and
- For the tilt servo, to use a greater range, the commanded servo angles should be twice the detected angle; i.e. the clamp bound of this servo should be set at 140°.

To determine the required orientation angles of the aircraft, a MPU6050 inertial measurement unit is utilised. From the detected components of acceleration via the accelerometer $\mathbf{a} = \left(a_x, a_y, a_z\right)$, it can be shown that the roll and pitch angles are determined using the following formulae:

$$\phi = \text{atan2}\left(a_y, a_z\right) \tag{1}$$

$$\theta = \text{atan2}\left(-a_x, \sqrt{{a_y}^2 + {a_z}^2}\right) \tag{2}$$

$$\alpha = \sqrt{\theta^2 + \phi^2} \tag{3}$$

where atan2 is the four-quadrant arctangent function that will assign correct sign to the calculated angle. This function is found in the maths libraries of all programming languages (in C, including `math.h` in your file gives you access to atan2f which is called using the syntax illustrated by Eq. (1) and (2)). Your code **must have a subroutine which calculates these angles**.

Due to the noisy nature of real-world accelerometer readings; to disengage from a clamped to compliant force or from a compliant to free motion a deadband of 5° is applied on the limits. For example, transitioning from clamped to compliant in the pitch axis requires the detected angle to be $\pm 45^\circ$ (depending on which limit you were at) instead of the $\pm 50^\circ$ to enter the clamped zone.

The Wokwi setup is illustrated in Fig. 1 and Table 1 provides information on the external hardware and pin configurations for the `Nucleo C031C6`. You should poll the I2C bus to check the MPU6050 at the same rate as your servo update.

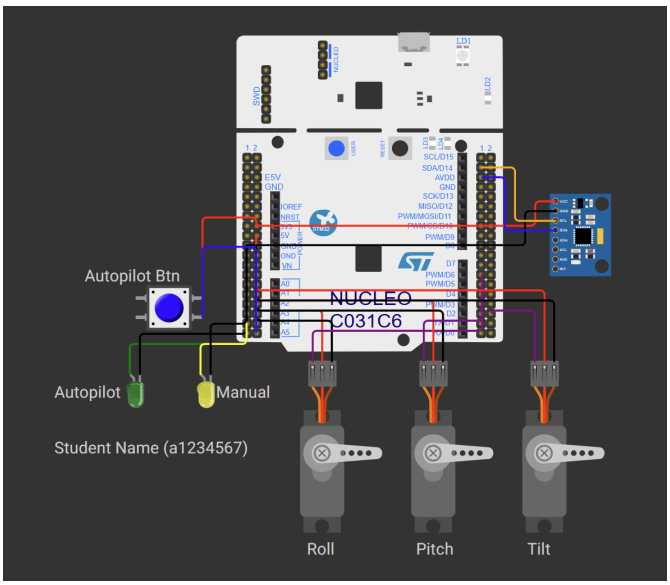


Figure 1: Wokwi Emulator setup for the flight control haptic system

Hardware	Signal Name	Pin	Description
Haptic Servos	`Roll`	`PB4`	PWM (50 Hz)
	`Pitch`	`PB5`	PWM (50 Hz)
	`Tilt`	`PB0`	PWM (50 Hz)
Indicator LEDs	`Autopilot`	`PA2`	Output
	`Manual`	`PB11`	Output
MPU6050 IMU	`SCL`	`PB8`	I2C Serial Clock
	`SDA`	`PB9`	I2C Serial Data
Autopilot Button	`Autopilot Btn`	`PA12`	Input (Pull-Down)

Table 1: STM32C031C6 Pin Allocations and Hardware Mappings

Your code must implement a finite state machine based on the expected actions listed in the brief. It is highly suggested that you consider having multiple state machines, i.e. one for individual axis control (clever code would re-use one FSM for each axis) and another for the unified tilt control. You are free to use any code provided to you in lectures, practicals or workshops. No additional code is given; this is a **20% assignment**.

Assignment Deliverables

A brief report (word count: 1500-2000 words) which outlines your embedded code solution is required. This should detail reasoning behind your code logic, i.e. custom subroutines, finite state machine implementation and pertinent register configurations. The primary aspect of your report should be diagrams of your FSM(s) which shows the actions in each state and the triggers for state transitions, how this links to the implemented code and important results which highlight correct state transitions. Given the low fidelity of accelerometer steps, you can show the various states triggering (i.e. free motion to compliant to clamped) by slowly changing the slider(s) of the accelerometer within your recorded video. That is, this demonstration does not need to be mechanically feasible, and is simply used to show the FSM transitions work on a high level.

With results provided in your report, you should document various combinations of accelerometer triples $\mathbf{a} = (a_x, a_y, a_z)$ that trigger single axis entry into the compliant and impenetrable zones, along with similar triples for the unified tilt value. How you choose these values is up to you, i.e. you can utilise trial and error with the slider or you may want to create a series of plots using Eq. (1)-(3) by fixing acceleration values in a specific axis/axes and then finding the dependent values that meets your thresholds. **Note, the assumption is that this haptic controller is enforced during cruise period, i.e. constant velocity. This means that the magnitude of the acceleration vector must be 1g, i.e.** $\|\mathbf{a}\| = \sqrt{a_x^2 + a_y^2 + a_z^2} = 1g$**, for the specific test cases you document (you are allowed reasonable margin of error due to the Wokwi MPU6050 fidelity issue)**. The more methodical you are about this, the more favourable your grade. **At a minimum**, your results should show values that provide compliant haptic feedback for roll only; for pitch only; and for tilt only (i.e. roll and pitch are in free motion). Once again, the more thorough your results (i.e. linked to your FSM conditions); the more favourable your grade.

All diagrams must be created via software, i.e. no hand-drawn FSMs. **You do not need to include a flow chart in this report**. Along with the report, you must include all your code files. All code written by yourself must be fully commented. If you have utilised code from lectures/pracs; please acknowledge this in your report and briefly as comments in your code files. **Providing the final code in report is not required; however, if you do please utilise appendices for major blocks of code. All code that does appear in the report must be properly formatted, i.e. no screenshots. Failure to do either of these will result in penalties.**

Accompanying this, please take a screen recording of your code working in Wokwi (a program such as Snipping Tool on Windows can do this). The video should match with the results of your report and show the state transitions described by your FSM diagrams. **Please add your name and student ID to the Wokwi diagram.json file (line 7):**

```
{ "type": "wokwi-text", "id": "lbl_name", "top": -170, "left": -100, "attrs": { "text": "Student Name (
    a1234567)" } },
```

The rubric with breakdown of marks is available on the myLearning page. **As detailed on the myLearning page, the use of AI is allowed for editing purposes such as spelling and grammar only. The use of AI to generate code is prohibited.**

Tips

- Utilise the I2C code provided to you in Lecture 8. You will not need to write any I2C wrapper functions; the provided code will work, you just need to interpret the register map of the MPU6050 for the data you require to calculate the orientation angles;
- Lecture 9 provides an example of how to write a subroutine for calculating floating point outputs using functions present in `math.h`. To implement Eq. (1)-(3), you will need to utilise `atan2f(y,x)` and `sqrtf(x)` where `x,y` are inputs to the functions. If you are required to square a variable, i.e. x^2, simply use the syntax `x*x`; and
- Develop your finite state machine(s) in stages; do not try to solve full functionality from the outset. That is, focus on a single axis and the transitions between regions before adding in autopilot pushbutton triggering etc.